



BANK OF CLI

AAREJ SYED

PROJECT OVERVIEW

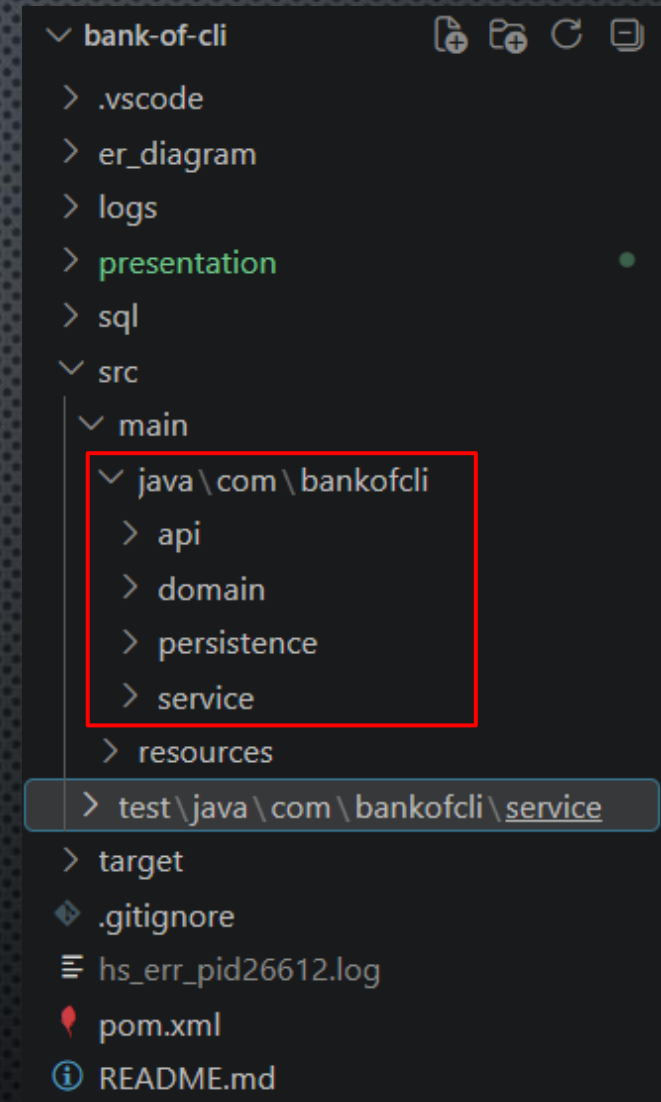
- BANK OF CLI'S *CORE LEDGER* IS A PROTOTYPE BANKING APPLICATION THAT RUNS IN THE COMMAND TERMINAL
- THE CORE LEDGER PROVIDES:
 - ACCOUNT REGISTRATION AND LOGIN
 - ACCOUNT BALANCE CHECKING
 - BANK TRANSACTIONS: DEPOSIT, WITHDRAWAL, AND TRANSFER
 - BANK TRANSACTION HISTORY
 - SYSTEM LOGGING

TECHNOLOGY STACK

- JAVA: PROGRAMMING LANGUAGE
- MAVEN: BUILD TOOL
- POSTGRESQL: DATABASE LANGUAGE
- JDBC: DATABASE COMMUNICATION
- JUNIT 5 AND MOCKITO: TESTING
- SLF4J/LOGBACK: LOGGING

PROJECT STRUCTURE

- THE APPLICATION'S JAVA CODEBASE IS DIVIDED AMONG FOUR SUBDIRECTORIES:
 - API/
 - SERVICE/
 - DOMAIN/
 - PERSISTENCE/
- THIS ESTABLISHES A MULTILAYER SOFTWARE ARCHITECTURE WHERE DIFFERENT LAYERS HAVE DIFFERENT RESPONSIBILITIES



MULTILAYER ARCHITECTURE



API: Displays information to the user and handles user input



Service: Handles business logic; the “brain” of the application



Domain: Provides abstraction over database tables and records



Persistence: Talks to the database and “Translates” between Java and SQL

OBJECT-ORIENTED PROGRAMMING

AND HOW THE BANK OF CLI CORE LEDGER APPLIES IT

OOP: ENCAPSULATION

- CLASSES HIDE PRIVATE MEMBERS AND EXPOSE THEM TO THE OUTSIDE WORLD THROUGH PUBLIC GETTER AND SETTER METHODS

```
public class Account {
    private long id;
    private String pin;
    private BigDecimal balance;

    // New account to be registered
    public Account(String pin) {
        this.pin = pin;
    }

    // Existing account that is already registered
    public Account(long id, String pin, BigDecimal balance) {
        this.id = id;
        this.pin = pin;
        this.balance = balance;
    }

    public long getId() { return id; }
    public String getPin() { return pin; }
    public BigDecimal getBalance() { return balance; }

    public void setId(long id) { this.id = id; }
    public void setPin(String pin) { this.pin = pin; }
    public void setBalance(BigDecimal balance) { this.balance = balance; }
}
```

OOP: ABSTRACTION

```
public class Main {  
    Run | Debug  
    public static void main(String[] args) {  
        BankDao bankDao = new BankDaoImpl();  
        BankService bankService = new BankServiceImpl(bankDao);  
        new BankRepl(bankService).run();  
    }  
}
```

▼ persistence

└ BankDao.java

└ BankDaoImpl.java

└ ConnectionFactory.java

▼ service

└ BankService.java

└ BankServiceImpl.java

- THE SERVICE LAYER AND THE DAO (PERSISTENCE LAYER) ARE NOT JUST CONCRETE CLASSES
- EACH HAS AN INTERFACE AND A CONCRETE IMPLEMENTATION
- WE ACCESS THESE OBJECTS THROUGH INTERFACES RATHER THAN DIRECTLY THROUGH THE CONCRETION
- THIS MEANS WE DON'T NEED TO CONCERN OURSELVES WITH THE IMPLEMENTATION OF THESE CLASSES WHEN USING THEM

OOP: ABSTRACTION (CONTINUED)

```
package com.bankofcli.service;

import java.math.BigDecimal;
import java.util.List;

import com.bankofcli.domain.BankTransaction;

// Service layer interface
public interface BankService {
    // Secure access
    long registerAccount(String pin);
    boolean logInToAccount(long accountId, String pin);
    void logOutOfAccount();
    boolean isLoggedIn();

    // Account management
    long getAccountId();
    boolean accountExists(long accountId);

    // Balance management
    BigDecimal getAccountBalance();

    // Transaction engine
    void deposit(BigDecimal amount);
    void withdraw(BigDecimal amount);
    void transfer(BigDecimal amount, long destinationAccountId);

    // Audit trail
    List<BankTransaction> getBankTransactionHistory();
}
```

```
// Service layer concretion
public class BankServiceImpl implements BankService {
    private final BankDao bankDao;
    private Account loggedInAccount;
    private static Logger logger;

    // Constructor: requires Bank DAO
    public BankServiceImpl(BankDao bankDao) { ...

    // Register new account
    @Override
    public long registerAccount(String pin) throws IllegalArgumentException { ...

    // Log in to account
    @Override
    public boolean logInToAccount(long accountId, String pin) throws NoSuchElementException { ...

    // Log out of account
    @Override
    public void logOutOfAccount() throws RuntimeException { ...

    // Get account balance
    @Override
    public BigDecimal getAccountBalance() throws RuntimeException { ...

    // Deposit money into account
    @Override
    public void deposit(BigDecimal amount) throws RuntimeException, IllegalArgumentException { ...

    // Withdraw money from account
    @Override
    public void withdraw(BigDecimal amount) throws RuntimeException, IllegalArgumentException { ...

    // HELPER METHOD: Perform standard bank transaction
    private void standardBankTransaction(BankTransactionType type, BigDecimal amount) { ...
}
```

OOP: POLYMORPHISM

- THE CONCRETIONS OF THE SERVICE AND PERSISTENCE INTERFACES OVERRIDE THE INTERFACE METHODS AND PROVIDE THEIR OWN FUNCTIONALITY

```
// Insert new Account
@Override
public Account insertAccount(Account account) {
    // Create connection
    try (Connection connection = ConnectionFactory.getConnectionFactory().getConnection()) {
        // Set auto-commit to false
        connection.setAutoCommit(autoCommit: false);

        // Run SQL prepared statement to insert Account
        try (PreparedStatement statement = connection.prepareStatement(INSERT_ACCOUNT_SQL)) { ...

        // If Account insertion fails, roll back and throw error
        catch (SQLException e) { ...

    }

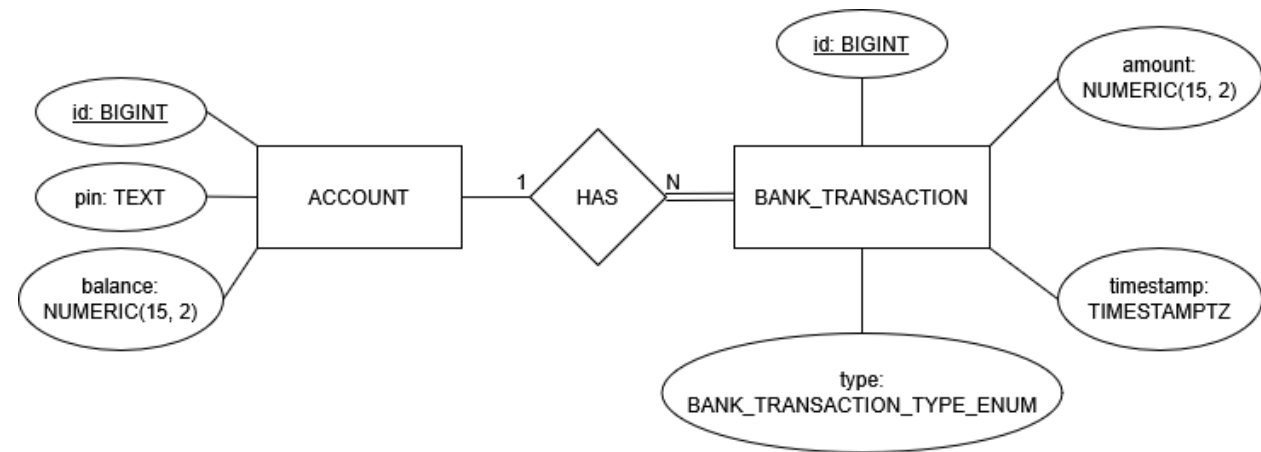
    // Catch connection failure
    catch (SQLException e) { throw databaseConnectionError(e); }
}
```

DATABASE DESIGN

HOW THE CORE LEDGER STORES, MANAGES, AND REPRESENTS DATA

DATABASE DESIGN

- OUR DATABASE CONTAINS ACCOUNT AND BANK_TRANSACTION TABLES
- IT ALSO DEFINES THE CUSTOM ENUM TYPE, BANK_TRANSACTION_TYPE_ENUM, WITH VALUES 'DEPOSIT', 'WITHDRAWAL', 'TRANSFER_IN', AND 'TRANSFER_OUT'
- ACCOUNT AND BANK_TRANSACTION HAVE A 1:N RELATIONSHIP
- ACCOUNTS DO NOT NEED TO HAVE BANK_TRANSACTIONS, BUT BANK_TRANSACTIONS MUST BE LINKED TO ACCOUNTS



DATABASE DESIGN

- CHECKS
 - PIN MUST BE A 4-DIGIT STRING
 - BALANCE CANNOT BE NEGATIVE
 - AMOUNT OF MONEY INVOLVED IN A TRANSACTION MUST BE POSITIVE
- TO PRESERVE TRANSACTION HISTORY, WE PREVENT ACCOUNT DELETION IF IT HAS ASSOCIATED TRANSACTIONS
- INDEXING OPTIMIZES BANK_TRANSACTION LOOKUPS

```
-- Create Account table
CREATE TABLE account (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  pin TEXT NOT NULL
    CHECK(pin ~ '^[0-9]{4}$'),
  balance NUMERIC(15, 2) NOT NULL DEFAULT 0.00
    CHECK(balance >= 0.00)
);

-- Create Bank_Transaction_Type_Enum type
CREATE TYPE bank_transaction_type_enum AS ENUM (
  'deposit',
  'withdrawal',
  'transfer_in',
  'transfer_out'
);

-- Create Bank_Transaction table
CREATE TABLE bank_transaction (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  account_id BIGINT NOT NULL REFERENCES account(id)
    ON DELETE RESTRICT,
  amount NUMERIC(15, 2) NOT NULL
    CHECK(amount > 0.00),
  timestamp TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  type bank_transaction_type_enum NOT NULL
);

-- Optimize Bank_Transaction lookups by account_id and chronological order
CREATE INDEX bank_transaction_account_id_timestamp_idx
ON bank_transaction(account_id, timestamp DESC);
```

-- Account PIN must be 4 digits long

-- Account balance cannot be negative

-- We block attempts to delete associated account

-- Bank_Transaction dollar amount must be positive

DEFENSIVE DESIGN

HOW THE CORE LEDGER ANTICIPATES AND PROTECTS AGAINST UNEXPECTED
SCENARIOS

DEFENSIVE DESIGN: INPUT VALIDATION

THE CORE LEDGER PERFORMS QUICK
VALIDATIONS OF USER INPUT:

- INTEGER COMMANDS
- NUMERIC INPUT
- `LONG`-TYPE IDs

```
// Get valid integer command from user
// Verifies input is integer and that it corresponds to one of the provided commands
private int getValidIntCommandFromUser(List<String> commands) {
    int command;

    // Get valid user command
    while (true) {
        printCommands(commands);

        // Get command
        try {
            command = Integer.parseInt(userInput());
        }

        // Ensure command is integer
        catch (NumberFormatException e) {
            System.out.println(x: x: "Command input must be an integer.");
            System.out.println();
            continue;
        }

        // Ensure command is valid
        if (command >= 0 && command < commands.size()) { break; }
        else {
            System.out.println(x: x: "Command input must be a valid command.");
            System.out.println();
        }
    }

    return command;
}
```

```
// Get amount of money from user
private BigDecimal getAmountFromUser(String action) {
    System.out.println("Amount to " + action + ":");
    String amountString = userInput();

    // Convert amount to BigDecimal
    BigDecimal amount;
    try {
        amount = new BigDecimal(amountString);
    }

    // Notify user and end process if amount is not a valid number
    catch (NumberFormatException e) {
        System.out.println(x: x: "Input must be a valid number of format XXX.XX.");
        System.out.println();
        return null;
    }

    // Notify user and end process if amount is not positive or has more than 2 decimal places
    if (amount.signum() <= 0 || amount.scale() > 2) {
        System.out.println(x: x: "Amount of money must be positive and must have at most 2 decimal places.");
        System.out.println();
        return null;
    }

    return amount;
}
```

```
// Get ID (account or bank transaction) from user
private long getIdFromUser(String prompt) {
    System.out.println(prompt);
    String idString = userInput();

    // Convert ID to long
    long id;
    try { id = Long.parseLong(idString); }

    // Notify user and end process if ID is not a valid integer
    catch (NumberFormatException e) {
        System.out.println(x: x: "Input must be a valid integer.");
        System.out.println();
        return -1;
    }

    return id;
}
```

DEFENSIVE DESIGN: INTERNAL RULES

THE CORE LEDGER ALSO ENFORCES INTERNAL RULES AFTER BASIC INPUT VALIDATION, SUCH AS:

- PIN VALIDATION
- VERIFICATION OF USER LOG-IN STATUS
- NONEXISTENT-ACCOUNT HANDLING
- SAME-ACCOUNT TRANSFER PREVENTION
- OVERDRAFT PREVENTION

DEFENSIVE DESIGN: INTERNAL RULES (CONTINUED)

```
if (!pin.matches(regex: regex: "\\d{4}")) {  
    logger.error(msg: msg: "User attempted to register account with invalid PIN format");  
    throw new IllegalArgumentException(s: s: "PIN must consist of four digits");  
}
```

```
// Transfer money to another account
```

```
@Override
```

```
public void transfer(BigDecimal amount, long destinationAccountId) throws RuntimeException, NoSuchElementException, IllegalArgumentException {
```

```
    if (!isLoggedIn()) { throw new RuntimeException(message: message: "Cannot transfer money while not logged into account"); }
```

```
    Account destinationAccount = bankDao.selectAccountById(destinationAccountId).orElseThrow(  
        () -> new NoSuchElementException("Cannot transfer money to non-existent account " + destinationAccountId)
```

```
    );
```

```
    if (destinationAccountId == loggedInAccount.getId()) { throw new IllegalArgumentException(s: s: "Cannot transfer money to the same account."); }
```

```
    if (amount.signum() <= 0) { throw new IllegalArgumentException(s: s: "Amount of money transferred must be positive"); }
```

```
    if (amount.compareTo(loggedInAccount.getBalance()) > 0) { throw new IllegalArgumentException(s: s: "Amount of money transferred cannot exceed account balance"); }
```

```
    logger.info(format: format: "Transferring ${} from account ID {} to account ID {}", amount, loggedInAccount.getId(), destinationAccount.getId());
```

```
    loggedInAccount.setBalance(loggedInAccount.getBalance().subtract(amount));
```

```
    destinationAccount.setBalance(destinationAccount.getBalance().add(amount));
```

```
    transferBankTransaction(amount, destinationAccount);
```

```
}
```

COMMUNICATING WITH THE DATABASE

SECURITY

- CORE LEDGER USES PREPARED STATEMENTS WHEN COMMUNICATING WITH THE DATABASE
- BY SEPARATING THE DATA FROM THE QUERY ITSELF, CORE LEDGER PREVENTS SQL INJECTIONS

```
// Select Account by ID
@Override
public Optional<Account> selectAccountById(long id) {
    // Create connection
    try (Connection connection = ConnectionFactory.getConnectionFactory().getConnection()) {
        // Run SQL prepared statement to select Account by ID
        try (PreparedStatement statement = connection.prepareStatement(SELECT_ACCOUNT_BY_ID_SQL)) {
            // Set parameters of prepared statement
            statement.setLong(parameterIndex: 1, id);

            // Execute statement and capture result set
            try (ResultSet resultSet = statement.executeQuery()) {
                // If result exists, map to Account object and return
                if (resultSet.next()) {
                    return Optional.of(mapAccount(resultSet));
                }

                // If result does not exist, return null
                else { return Optional.empty(); }
            }
        }

        // If Account selection fails, throw error
        catch (SQLException e) {
            throw databaseError("Could not find Account with ID: " + id, e);
        }
    }

    // Catch connection failure
    catch (SQLException e) { throw databaseConnectionError(e); }
}
```

DATABASE TRANSACTIONS

- MULTI-STEP OPERATIONS (E.G. BANK TRANSACTIONS) ARE BUNDLED INTO ONE DATABASE TRANSACTION
- THIS ENSURES OPERATION IS ATOMIC
- PERSISTENCE LAYER COMMITS A TRANSACTION IF ALL STEPS WERE SUCCESSFUL
- IF EVEN ONE STEP FAILS, PERSISTENCE LAYER ROLLS BACK COMMIT

```
// Set auto-commit to false
connection.setAutoCommit(autoCommit: autoCommit: false);

try {
    // Update existing Accounts and insert new transfer Bank_Transactions
    this.updateAccount(connection, sourceAccount);
    this.updateAccount(connection, destinationAccount);
    this.insertBankTransaction(connection, sourceBankTransaction);
    this.insertBankTransaction(connection, destinationBankTransaction);

    // Commit changes and return updated BankTransaction objects
    connection.commit();
    return new BankTransaction[] {sourceBankTransaction, destinationBankTransaction};
}

// If helper actions fail, roll back and re-throw error
catch (RuntimeException e) {
    try {
        connection.rollback();
    }
    catch (SQLException rollbackException) {
        e.addSuppressed(rollbackException);
    }

    throw e;
}
```

LOGGING AND ERRORS

- CORE LEDGER RECORDS APPLICATION ACTIONS IN A LOG FILE
- WHEN ERRORS ARISE, THE APPLICATION DISPLAYS A USER-FRIENDLY DESCRIPTION OF THE ERROR

```
// Register new account
@Override
public long registerAccount(String pin) throws IllegalArgumentException {
    if (!pin.matches(regex: regex: "\\d{4}")) {
        logger.error(msg: msg: "User attempted to register account with invalid PIN format");
        throw new IllegalArgumentException(s: s: "PIN must consist of four digits");
    }

    Account newAccount = new Account(pin);
    bankDao.insertAccount(newAccount);
    logger.info(format: format: "Registered new account with ID: {}", newAccount.getId());
    return newAccount.getId();
}
```

TESTING

- CORE LEDGER USES JUNIT 5 AND MOCKITO FOR POSITIVE AND NEGATIVE TESTS, VERIFYING BOTH THAT A METHOD WORKS WITH VALID INPUT AND THAT IT PROPERLY HANDLES INVALID INPUT
 - JUNIT 5 ENABLES UNIT TESTS AND MOCKITO ENABLES THE MOCKING OF OBJECTS
 - THIS MEANS WE CAN USE A MOCK DAO IN UNIT TESTS WITHOUT NEEDING TO TEST THE DATABASE CONNECTION

TESTING (CONTINUED)

```
// Test for successful withdrawal with sufficient funds
@Test
void withdrawWithSufficientFundsDecreasesBalance() {
    // Create an account with an initial balance of 51.00
    Account account = new Account(
        id: id: 1L,
        pin: pin: "1111",
        new BigDecimal(val: val: "51.00")
    );

    // Arrange for mock BankDao to return the account when queried by ID
    when(bankDao.selectAccountById(accountId: accountId: 1L))
        .thenReturn(Optional.of(account));

    // Log in to the account and withdraw 40.00
    bankService.logInToAccount(accountId: accountId: 1L, pin: pin: "1111");
    bankService.withdraw(new BigDecimal(val: val: "40.00"));

    // Assert that the balance is decreased by the withdrawal amount
    assertEquals(
        new BigDecimal(val: val: "11.00"),
        account.getBalance()
    );
}
```

```
// Set up mock BankDao and BankServiceImpl before each test
@BeforeEach
void setUp() {
    bankDao = mock(classToMock: classToMock: BankDao.class);
    bankService = new BankServiceImpl(bankDao);
}
```

```
// Test for withdrawal with insufficient funds, expecting an exception
@Test
void withdrawWithInsufficientFundsThrowsException() {
    // Create an account with an initial balance of 51.00
    Account account = new Account(
        id: id: 1L,
        pin: pin: "1111",
        new BigDecimal(val: val: "51.00")
    );

    // Arrange for mock BankDao to return the account when queried by ID
    when(bankDao.selectAccountById(accountId: accountId: 1L))
        .thenReturn(Optional.of(account));

    // Log in to the account
    bankService.logInToAccount(accountId: accountId: 1L, pin: pin: "1111");

    // Attempt to withdraw 57.00, which exceeds the account balance, and assert that an IllegalArgumentException is thrown
    assertThrows(
        IllegalArgumentException.class,
        () -> bankService.withdraw(new BigDecimal(val: val: "57.00"))
    );

    // Assert that the balance remains unchanged after the failed withdrawal attempt
    assertEquals(
        new BigDecimal(val: val: "51.00"),
        account.getBalance()
    );
}
```

LIVE DEMONSTRATION AND Q&A